K-Nearest Neighbours

Github	https://github.com/jacksonjgee
Linkedin	linkedin.com/in/jacksonjgee/

Table of Contents

Table of Contents
Introduction
Mathematic Theory
Python Implementation

Introduction

K-Nearest Neighours, or KNN, is a classic supervised machine learning model commonly used for classifiction. It classifies a instance based on the majority class of k closest training instances.

For exampe: In an KNN model that uses $k = 5$, we classify a new point based on the classes of its closest 5 neighbours. If the closest neigbours are of the class: $\mathrm{Red}: 2$ and $\mathrm{Blue}: 3$ - then that new instance will be classified as Blue becasue that is the majority class of the 5 nearest neighbours.

Unlike models such as logistic regression or linear regression, KNN does not learn parameters during training. Instead, the training data itself is stored and used directly when making predictions. When a new instance is given to the model, KNN calculates the distance between the new instance and the training examples, identifies the k nearest neighbours, and assigns the majority class.

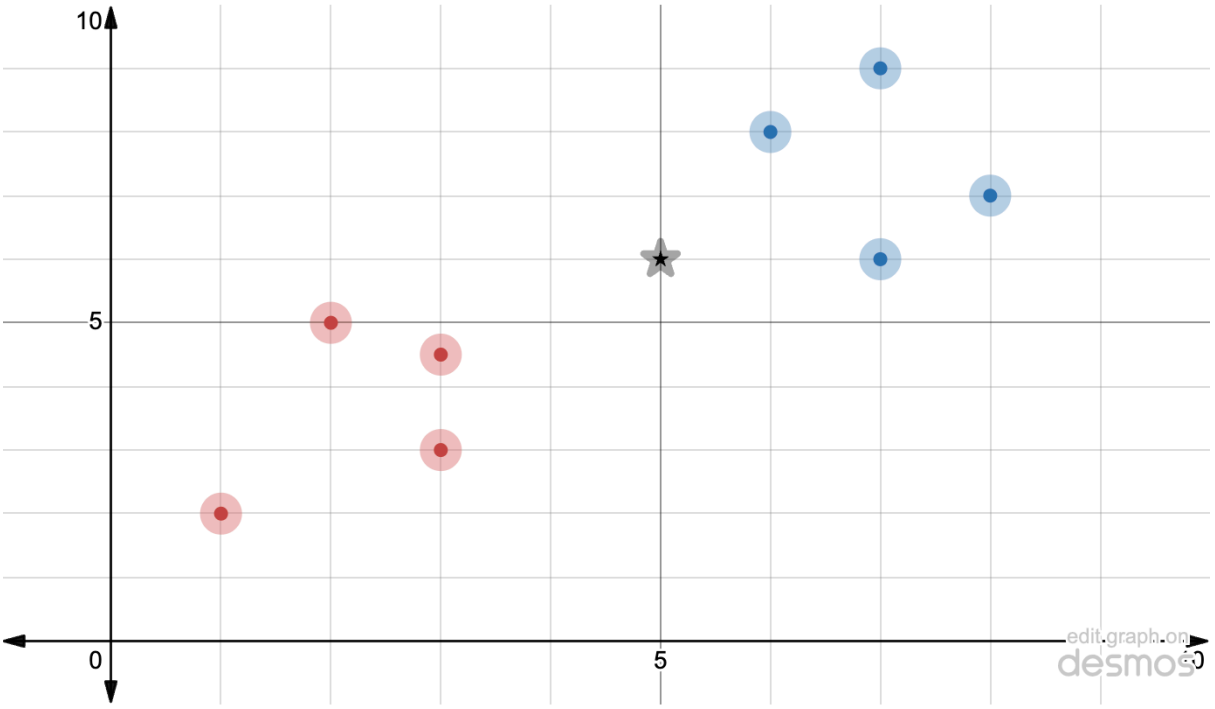
This means KNN is often described as a **lazy learning algorithm**, because most of the work happens during prediction rather than during training. However, we still usually split the data into training and testing sets so that we can evaluate how well the model performs on unseen data.

Mathematic Theory

Lets say we have the class with the following points:

$$\text{Red: } (3,3),\ (3,4.5),\ (1,2),\ (2,5)$$
$$\text{Blue: } (7,6),\ (6,8),\ (8,7),\ (7,9)$$

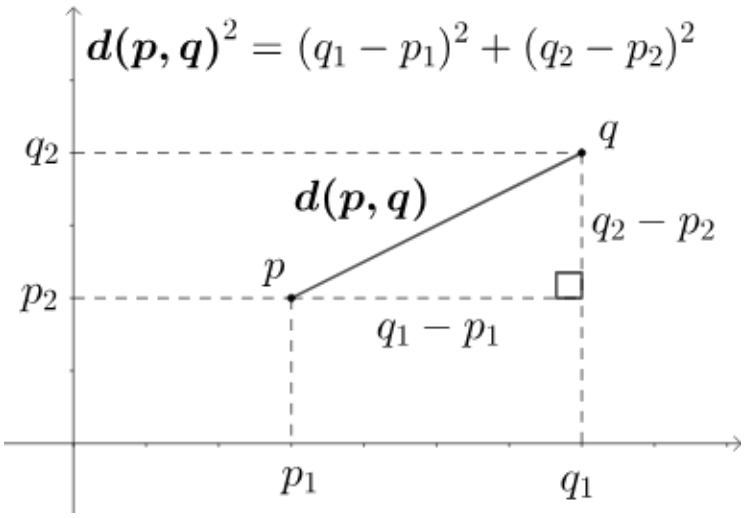
And we want to classify the new point $(5,6)$, represented by the star, in a KNN where $k = 3$.



We must first calculate the eclidean distance between the new point $(5,6)$ and all other points. To do this we use pythag:

$$c = \sqrt{a^2 + b^2}$$

Or more specifically:

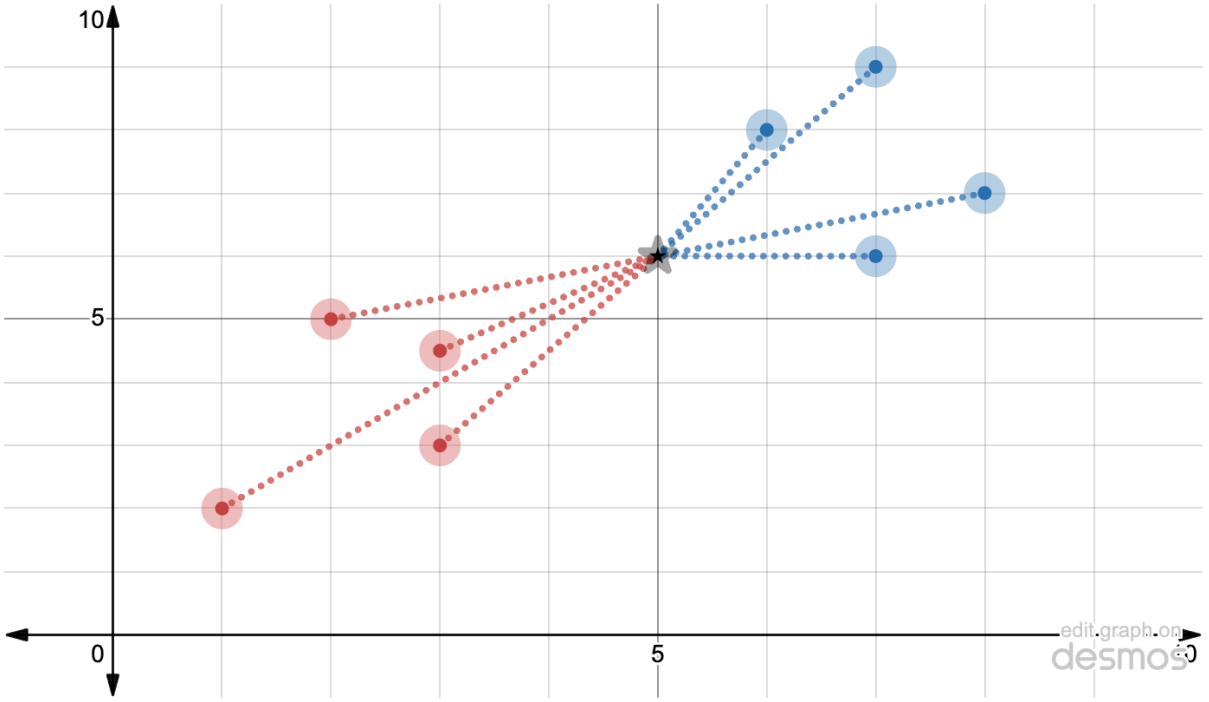


$$d(p,q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2}$$

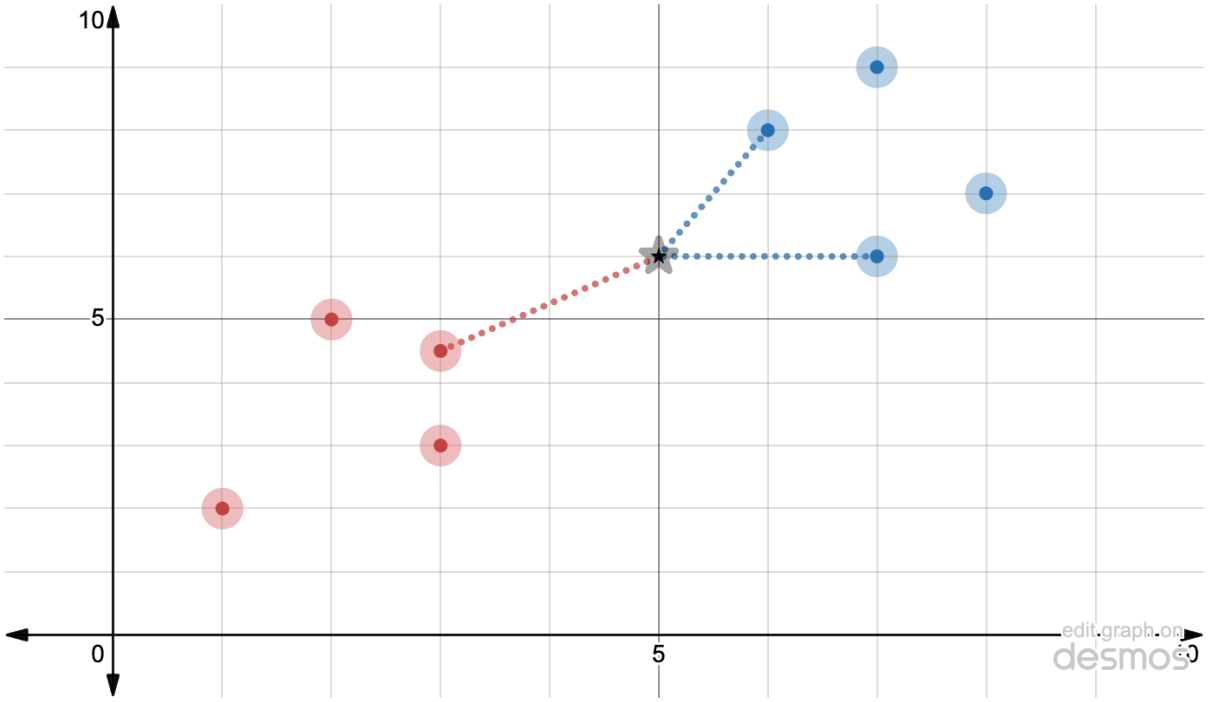
This has a general formula that can be used with any number of dimesions/features:

$$\boxed{d(p,q) = \sqrt{\sum_{i=1}^{n}(q_i - p_i)^2}}$$

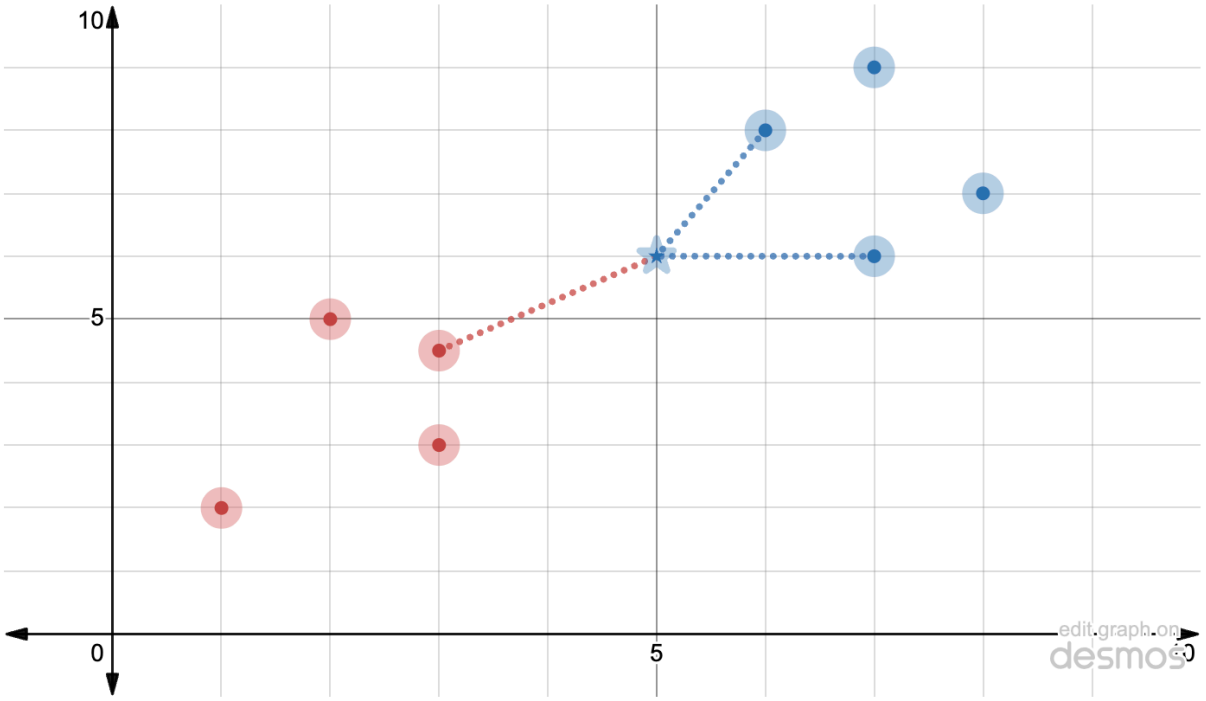
Where n is the dimensional degree, p is the new instance and q is a training instance. For our example, it might look like this after using the distance formula:



We then only keep the closest k neighbours, or the smallest k distances. Since $k = 3$ we have:



Now we can see that there are 2 blues points and 1 red point within our 3 nearest neighbours to the new point. Therefore we should classify the new point as Blue, because it is the majority class in our k nearest neighbour.



Note that it is important to have k be an odd number to handle tie-breakers where there are the same number of blue and red classes within the k nearest neighbours. Also note that since the general eucildean distance formula can work with any number of dimensions, it means we are not limited to $2D$. We can even use instances with 40 features or 40-Dimesnions.

Python Implementation

1. KNN Class and Training Data

```
class KNN:
    def __init__(self, k):
        self.k = k
        self.points = {
            "Red": [[1, 2], [3, 2], [1, 1], [4, 2], [2,
4]],
            "Blue": [[7, 8], [9, 6], [8, 6], [9, 8], [7.5,
8]]
        }
```

The model stores the training points and the chosen value of `k`. Since KNN is a lazy learning algorithm, there is no parameter training step. The stored points

are used directly when making a prediction.

2. Euclidean Distance

```
def euclidean_distance(self, p, q):
    return np.sqrt(np.sum(np.square(np.array(p) - np.array
(q))))
```

This function calculates the Euclidean distance between two points. The points are converted into NumPy arrays so that subtraction can be done element-wise. The differences are squared, summed, and square-rooted.

3. Finding the Nearest Neighbours

```
def fit(self, new_point):
    distances = []

    for category in self.points:
        for point in self.points[category]:
            distance = self.euclidean_distance(point, new_p
oint)
            distances.append([distance, category])

    nearest_neighbours = sorted(distances, key=lambda x: x
[0])[:self.k]
```

For every training point, the model calculates its distance from the new point and stores both the distance and the class label. The distances are then sorted from smallest to largest, and only the closest `k` neighbours are kept.

4. Majority Voting

```
categories = [neighbour[1] for neighbour in nearest_neighbo
urs]

result = Counter(categories).most_common(1)

return 1 if result[0][0] == "Red" else 0
```

After selecting the nearest neighbours, the model counts the class labels and returns the majority class. In this implementation, `Red` is represented as `1` and `Blue` is represented as `0`.

---

5. Predict Function

Include this, but explain that it is mainly for plotting compatibility:

```
def predict(self, X):
    predictions = []

    for point in X:
        prediction = self.fit(point)
        predictions.append(prediction)

    return np.array(predictions)
```

The `predict()` method allows the model to classify multiple points at once. This is useful for plotting decision boundaries, because the visualisation tool needs to classify many background points across the graph.
